

Analyzing the Role of AI-Assisted Programming in Modern Software Development

Submitted by,
E. A. A. B. Edirisinghe
Index No : s17233
Faculty of Science
University of Colombo

Table of Contents

Introduction

Task 1 – Conceptual Understanding

- 1.1 What is AI-Assisted Programming?
- 1.2 Types of AI Coding Tools
- 1.3 Advantages of AI-Assisted Programming in Software Development
- 1.4 Risks and Limitations of AI-Assisted Programming

Task 2 – Practical Implementation

- 2.1 Introduction
- 2.2 Problem Description
- 2.3 Functional Requirements
- 2.4 Non-Functional Requirements
- 2.5 System Design
- 2.6 UML Diagrams
- 2.7 Version A – Traditional Programming Approach
- 2.8 Version B – AI-Assisted Programming Approach
- 2.9 Comparison of Traditional and AI-Assisted Implementations

Task 3: Critical Analysis

- 3.1 How AI Changed the Coding Approach
- 3.2 Whether AI Improved or Reduced Understanding
- 3.3 Situations Where AI Should Not Be Used
- 3.4 Ethical Considerations in AI-Generated Code

Task 4: Conclusion

Introduction

Artificial Intelligence (AI) has rapidly become an integral part of modern software development, transforming the way developers design, implement, test, and maintain software systems. Recent advances in generative AI and large language models have enabled the development of intelligent programming assistants capable of generating source code, explaining complex programming concepts, identifying bugs, suggesting improvements, and automating repetitive development tasks. As a result, AI-assisted programming has become increasingly popular among both professional software engineers and students.

Traditionally, software development relied entirely on the programmer's knowledge, experience, and problem solving abilities. Developers were responsible for writing code manually, debugging errors, consulting documentation, and optimizing software through trial and error. While this approach strengthens programming skills and promotes a deeper understanding of software engineering principles, it can also be time consuming and prone to human error.

AI-assisted programming introduces a collaborative approach in which developers work alongside intelligent tools to improve productivity and software quality. Tools such as ChatGPT, GitHub Copilot and similar AI-powered coding assistants can generate code snippets, recommend algorithms, explain programming concepts, detect potential bugs and assist with documentation. Rather than replacing programmers, these tools serve as intelligent assistants that help developers complete tasks more efficiently while allowing them to focus on higher level problem solving and software design.

Despite these advantages, AI-assisted programming also presents several challenges. AI-generated code may contain logical errors, security vulnerabilities, inefficient implementations or fabricated information, requiring careful human verification. Excessive dependence on AI tools may also reduce programmers' critical thinking and problem solving skills, particularly among beginners who rely on generated solutions without understanding the underlying concepts. Furthermore, ethical concerns related to academic integrity, intellectual property, data privacy and software security continue to be important considerations when integrating AI into software development workflows.

This report explores the role of AI-assisted programming in modern software development by examining its concepts, tools, advantages, risks and ethical implications. A practical software application will be developed using both a traditional programming approach and an AI-assisted programming approach. The two implementations will then be compared in terms of development time, code quality, error handling, readability and maintainability. Finally, the report provides a critical evaluation of the impact of AI on programming education and the software industry while discussing the responsible use of AI-assisted development tools.

Task 1 – Conceptual Understanding

1.1 What is AI-Assisted Programming?

Artificial Intelligence (AI)-assisted programming refers to the use of AI-powered tools and machine learning models to support software developers throughout the software development lifecycle. These tools are designed to assist with tasks such as code generation, code completion, debugging, refactoring, documentation, testing and code review. Rather than replacing human programmers, AI-assisted programming aims to improve productivity, reduce repetitive work and enhance software quality by acting as an intelligent programming assistant.

Recent advances in generative AI and Large Language Models have significantly improved the capabilities of AI-assisted programming tools. By training on vast collections of programming languages, software documentation and open-source repositories, these models can understand natural language instructions and generate functional source code in multiple programming languages. Developers can describe a programming task in plain English, and the AI system can suggest algorithms, generate complete functions, explain existing code or identify possible errors.

AI-assisted programming has become increasingly common in both educational and industrial environments. Popular AI coding assistants include ChatGPT, GitHub Copilot, Amazon CodeWhisperer, Cursor and Tabnine. These tools integrate with modern Integrated Development Environments (IDEs) such as Visual Studio Code, IntelliJ IDEA and Android Studio to provide real time coding assistance. In professional software development, AI tools are used to accelerate application development, automate repetitive coding tasks, improve documentation and assist in debugging complex software systems.

Despite their impressive capabilities, AI coding assistants are intended to complement rather than replace software developers. Human programmers remain responsible for software architecture, algorithm selection, security validation, testing, debugging and ensuring that the generated code satisfies project requirements and coding standards.

1.2 Types of AI Coding Tools

AI-assisted programming encompasses a variety of intelligent tools, each designed to support different aspects of software development. The most common categories are described below.

1.2.1 Code Generation Tools

Code generation tools automatically generate source code from natural language descriptions or partially written programs. These systems can create individual functions, complete classes, algorithms, database queries and even entire application templates based on developer prompts.

For example, a developer may request an AI assistant to “generate a Java class for managing library books,” and the tool can produce the required class structure, constructors, methods, and documentation within seconds. This significantly reduces development time, especially for repetitive programming tasks.

Examples:

- ChatGPT
- GitHub Copilot
- Cursor

1.2.2 Intelligent Code Completion

Unlike traditional auto-complete systems that simply predict keywords, AI-powered code completion understands the programming context and suggests complete statements, methods, loops or entire functions while the developer types. These suggestions improve coding efficiency and reduce typing effort.

Modern IDEs integrate these capabilities to provide real time recommendations based on the current source code, imported libraries and project structure.

1.2.3 Debugging and Error Detection Tools

Debugging assistants analyze source code to identify syntax errors, logical mistakes, runtime exceptions and inefficient implementations. They often provide explanations of compiler messages and recommend possible solutions.

For instance, if a Java program throws a `NullPointerException`, an AI assistant can explain the cause, identify the problematic line of code, and suggest corrective measures.

These tools reduce debugging time and help developers understand programming errors more effectively.

1.2.4 Code Refactoring Tools

Refactoring tools improve the internal structure of software without changing its external behaviour. AI-assisted refactoring systems can recommend cleaner variable names, simplify complex methods, remove duplicate code, optimize loops and improve object oriented design.

Refactoring enhances software readability, maintainability, and long term scalability while reducing technical debt.

1.2.5 Documentation Generation Tools

Maintaining accurate documentation is an important but often neglected aspect of software development. AI tools can automatically generate comments, API documentation, user guides and README files by analyzing source code.

Automated documentation improves software maintainability and makes projects easier for other developers to understand and extend.

1.2.6 Test Case Generation

Some AI systems automatically generate unit tests and test cases based on existing source code. These tests help developers verify software functionality, detect regressions and improve software reliability.

Generated test cases can significantly reduce the effort required to establish comprehensive software testing procedures.

1.3 Advantages of AI-Assisted Programming in Software Development

The increasing adoption of AI-assisted programming has introduced numerous benefits for software developers, organizations and educational institutions.

Increased Productivity

One of the primary advantages of AI-assisted programming is the significant improvement in developer productivity. Routine programming tasks such as writing boilerplate code, generating constructors, creating CRUD operations and producing documentation can be completed within seconds, allowing developers to concentrate on system architecture and business logic.

Faster Software Development

AI-generated code accelerates software development by reducing the amount of manual coding required. This enables development teams to complete projects more quickly while shortening development cycles and improving time-to-market.

Improved Code Quality

AI tools often recommend coding best practices, identify potential bugs and suggest optimized implementations. They encourage cleaner code structures and can improve consistency throughout a software project.

Enhanced Learning and Skill Development

Students and beginner programmers can use AI assistants to understand unfamiliar programming concepts, algorithms and programming languages. AI explanations can serve as interactive learning resources that complement textbooks and classroom instruction.

Improved Debugging Efficiency

AI systems assist developers by explaining compiler errors, identifying logical mistakes and suggesting corrections. This reduces debugging time and helps developers resolve issues more efficiently.

Better Documentation

Automatically generated comments and documentation improve software maintainability and facilitate collaboration among development teams.

Support for Multiple Programming Languages

Many AI coding assistants support dozens of programming languages, enabling developers to transition between technologies more easily and learn new programming environments.

1.4 Risks and Limitations of AI-Assisted Programming

Although AI-assisted programming provides substantial benefits, it also introduces technical, ethical and practical challenges that developers must carefully consider.

Hallucinated or Incorrect Code

Large Language Models occasionally generate code that appears correct but contains logical errors, outdated APIs or entirely fabricated functions. These hallucinations may compile successfully while producing incorrect results, making human verification essential.

Over dependence on AI

Excessive reliance on AI tools may reduce programmers' problem solving abilities and critical thinking skills. Students who consistently accept generated code without understanding it may struggle to develop strong programming fundamentals.

Security Vulnerabilities

AI generated code may unintentionally introduce insecure programming practices, including SQL injection vulnerabilities, insecure authentication mechanisms, poor input validation or improper exception handling. Developers remain responsible for conducting thorough security reviews before deploying AI generated software.

Copyright and Intellectual Property Concerns

Since AI models are trained using large collections of publicly available source code, questions have been raised regarding software licensing, copyright ownership and attribution. Developers should ensure that generated code complies with applicable licensing requirements and organizational policies.

Privacy and Confidentiality

Submitting proprietary source code or confidential business information to cloud-based AI services may expose sensitive information to external systems. Organizations handling confidential data should carefully evaluate privacy policies before integrating AI coding assistants into their development workflows.

Lack of Contextual Understanding

Although AI models generate convincing code, they do not possess true understanding of project requirements, organizational objectives or business rules. Consequently, generated solutions may fail to address domain specific constraints or software architecture requirements.

Reliability and Accountability

AI systems cannot assume responsibility for software failures. The responsibility for validating correctness, testing functionality, ensuring security and maintaining software quality always remains with human developers.

Task 2 – Practical Implementation

2.1 Introduction

To evaluate the impact of AI-assisted programming on software development, a ***Library Management System*** was selected as the programming problem for this assignment. The project was implemented using two different development approaches:

- Version A: Traditional programming approach, where the application was designed, implemented and debugged manually without AI assistance.
- Version B: AI-assisted programming approach, where an AI coding assistant was used to support code generation, debugging, code explanation, and refactoring.

The objective of this comparison is to evaluate how AI-assisted programming affects software development efficiency, code quality, readability, maintainability and problem solving during implementation.

2.2 Problem Description

A Library Management System is designed to assist librarians in managing books within a small library. The system allows users to maintain a collection of books and perform common library operations through a menu-driven console interface.

The application enables users to:

- Add new books
- Display all books
- Search books by title
- Borrow books
- Return borrowed books
- Remove books from the library

Each book contains the following information:

- Book ID
- Book Title
- Author Name
- Availability Status (Available/Borrowed)

The system stores the collection of books in memory using Java collections, making it suitable for demonstrating object oriented programming concepts without requiring a database.

2.3 Functional Requirements

The Library Management System shall provide the following functional requirements:

- Add Book - The user shall be able to enter the details of a new book and add it to the library collection.
- Display Books - The system shall display all books currently stored in the library, together with their availability status.
- Search Book - The user shall be able to search for a book using its title.
- Borrow Book - The system shall allow a user to borrow a book if it is currently available.
- Return Book - The system shall allow a previously borrowed book to be returned to the library.
- Remove Book - The user shall be able to permanently remove a book from the collection.
- Exit - The application shall terminate safely when the user selects the exit option.

2.4 Non-Functional Requirements

The application should satisfy the following non-functional requirements:

- Simple and user-friendly interface.
- Fast response for all operations.
- Modular object oriented design.
- Readable and maintainable source code.
- Easy to extend with additional features in future versions.

2.5 System Design

The Library Management System follows an object oriented programming approach consisting of separate classes responsible for different tasks.

The main classes are:

Book - Stores the information related to individual books, including the book ID, title, author, and availability status.

Library - Maintains the collection of books and implements operations such as adding, removing, searching, borrowing, and returning books.

Main - Provides the menu-driven interface and handles user interaction.

Separating responsibilities into individual classes improves modularity, readability, and maintainability.

2.6 UML Diagrams

To better understand the software architecture and system behaviour, the following UML diagrams were prepared.

2.6.1 Use Case Diagram

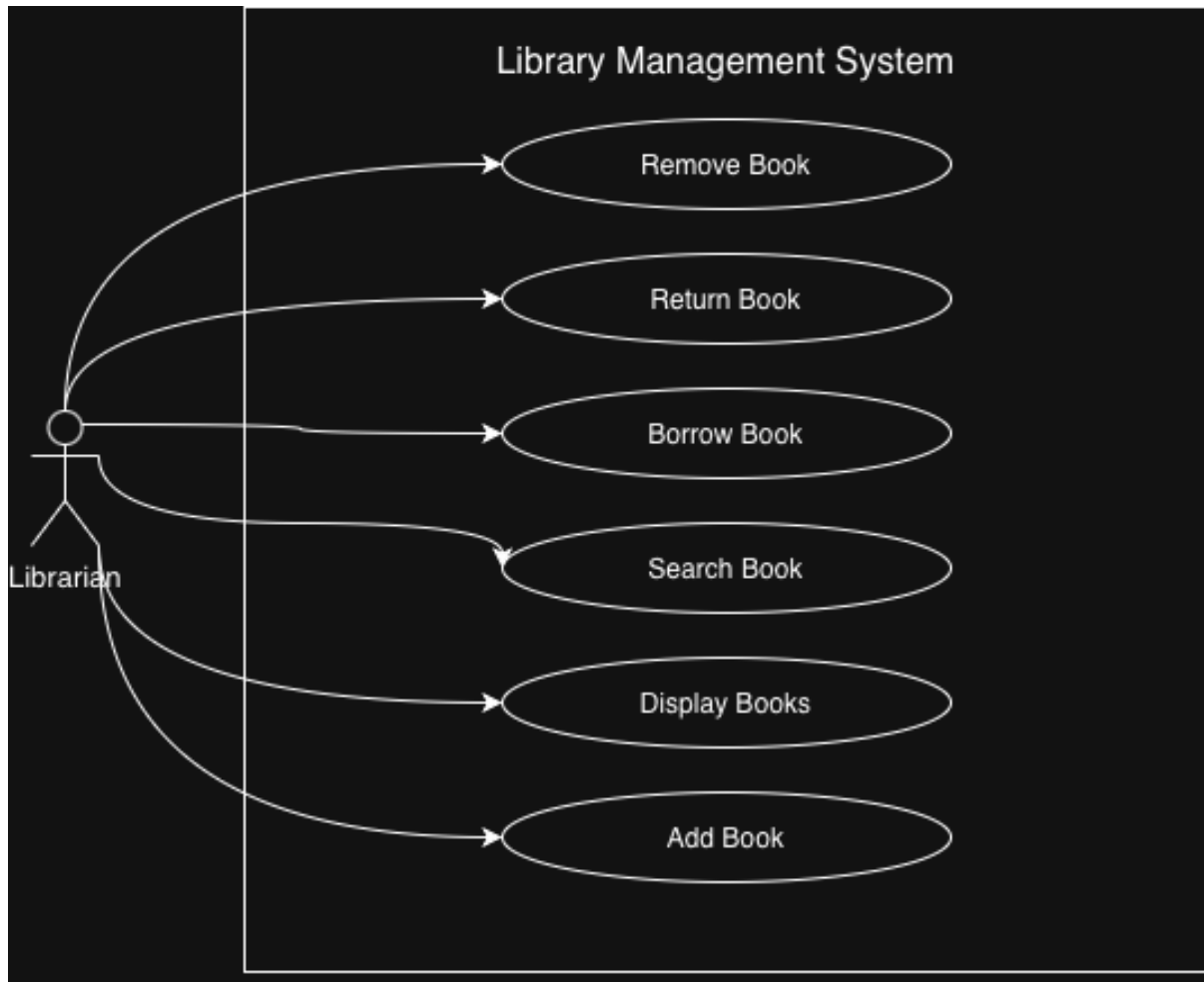


Figure 2.1 illustrates the interactions between the user and the Library Management System.

2.6.2 Class Diagram

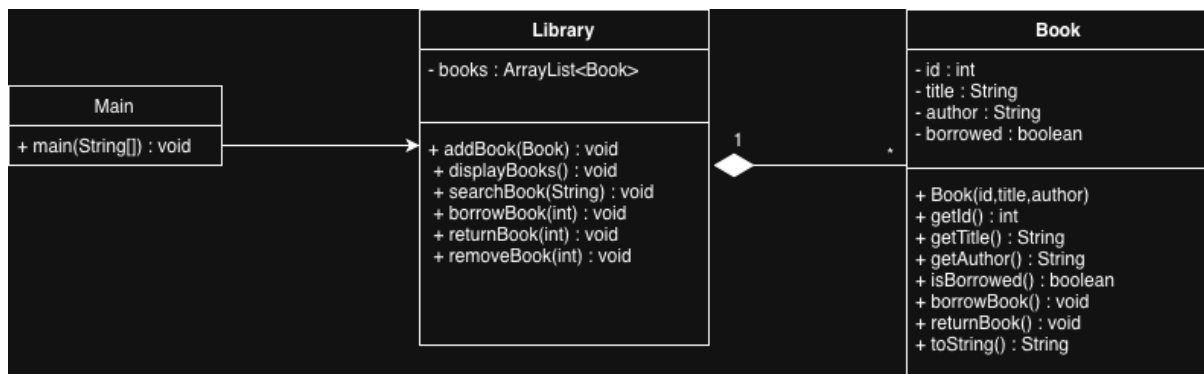


Figure 2.2 illustrates the classes used in the application together with their attributes, methods, and relationships.

2.6.3 Activity Diagram

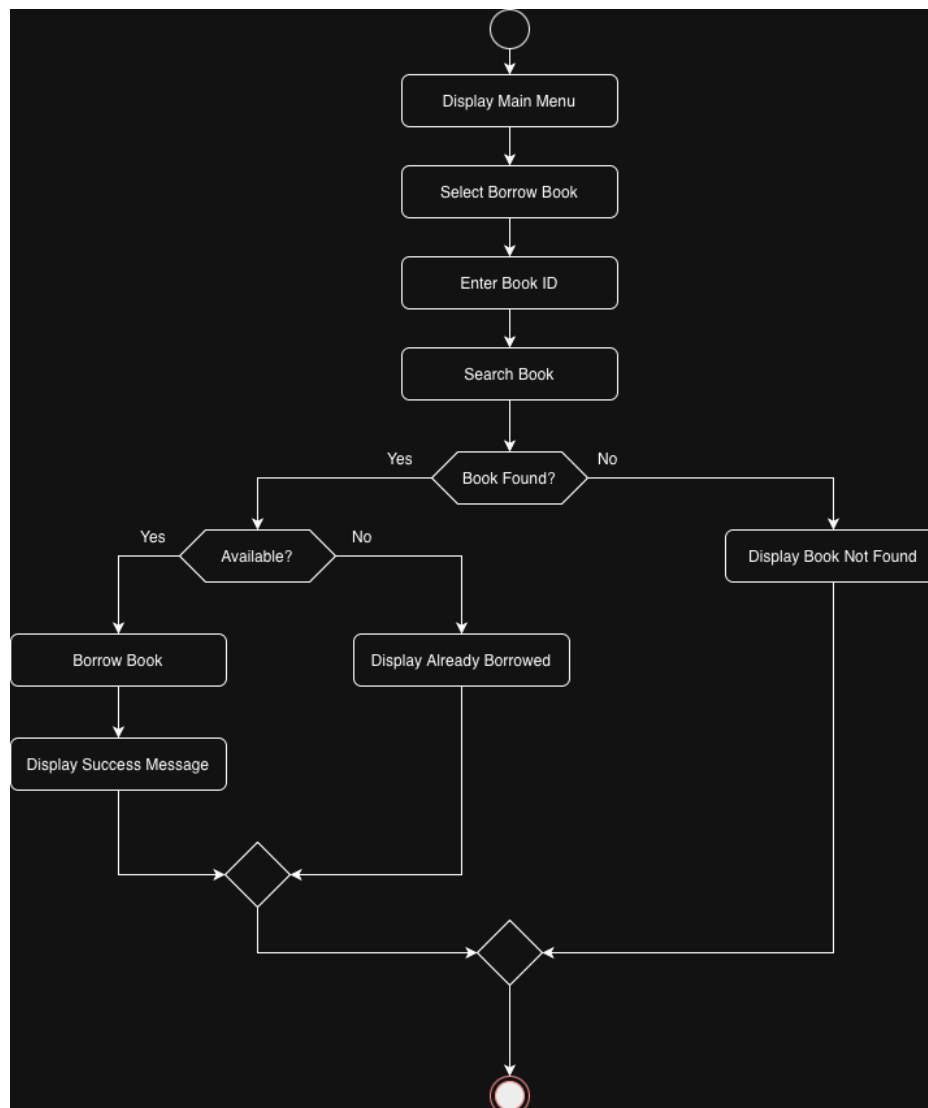


Figure 2.3 illustrates the workflow of performing library operations.

2.7 Version A – Traditional Programming Approach

In the first implementation, the Library Management System was developed entirely through traditional programming techniques without using any AI-assisted development tools.

The development process included:

- Designing the class structure manually.
- Writing all Java classes from scratch.
- Implementing the business logic independently.
- Testing each feature manually.
- Debugging compiler and runtime errors without AI assistance.

The traditional development process required continuous consultation of Java documentation, programming textbooks and online references whenever unfamiliar syntax or programming concepts were encountered.

The implementation followed a step by step software development process, beginning with class design, followed by method implementation, testing, debugging and code refinement.

2.8 Version B – AI-Assisted Programming Approach

The second implementation was developed with assistance from an AI programming tool(Gemini). Instead of manually writing every component, natural language prompts were used to request code generation, explanations, debugging assistance and code improvements.

Examples of AI assistance included

- Generating Java classes.
- Creating constructors and getter/setter methods.
- Generating search and borrow functions.
- Refactoring lengthy methods.
- Generating code comments.

Although Gemini produced working code for most components, manual modifications were still required. These included verifying the business logic, testing each function, improving console output and ensuring compatibility between the generated classes. The final implementation was therefore a collaboration between AI-generated suggestions and manual verification by the developer.

Compared with the traditional implementation, the AI-assisted version contained additional improvements such as input validation, descriptive comments, better console messages and enhanced code organization. These improvements reduced development effort while maintaining code readability and functionality.

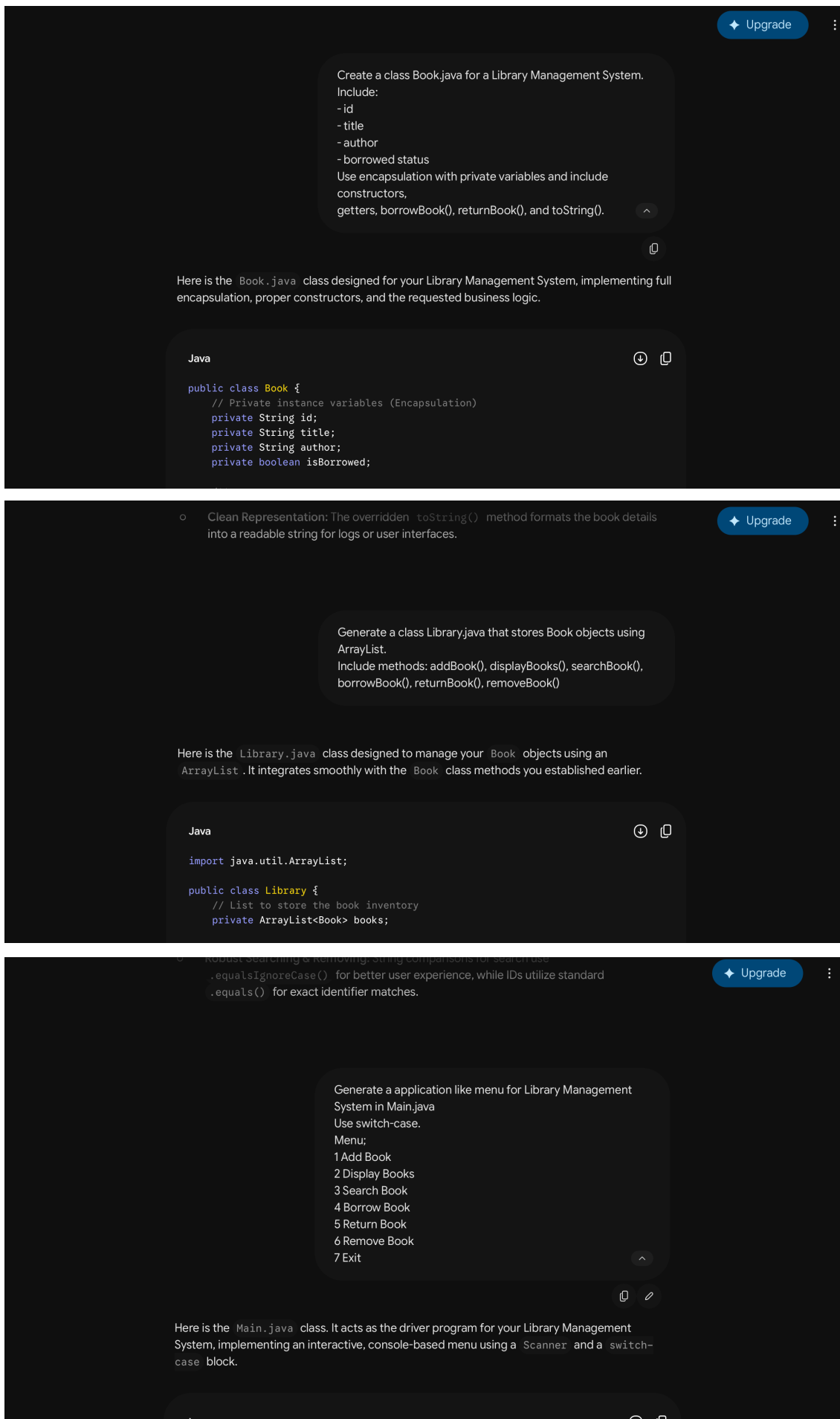


Figure 2.4 : Screenshots of the AI prompts used in Gemini

2.9 Comparison of Traditional and AI-Assisted Implementations

The Library Management System was implemented using two different development approaches: a traditional programming approach and an AI-assisted programming approach. The traditional implementation required the complete system to be designed and coded manually, whereas the AI-assisted implementation utilized ChatGPT to generate initial code, suggest improvements.

Evaluation Criterion	Traditional Programming Approach	AI-Assisted Programming Approach
Development Time	Required more time because all classes, methods, and logic were designed and implemented manually.	Development was significantly faster because AI generated the initial implementation of classes and methods
Code Quality	Produced functional code but required more manual effort to improve formatting, comments, and structure.	Generated well-structured code with descriptive variable names, comments, and consistent formatting.
Error Handling	Basic error handling was implemented manually, requiring additional effort to identify invalid inputs and logical errors.	Included improved input validation, clearer error messages, and suggestions for handling exceptional cases.
Readability and Maintainability	Code was readable but contained fewer comments and required additional refactoring to improve maintainability.	The generated code included JavaDoc comments, meaningful method names, and a consistent coding style, making it easier to understand, maintain, and extend after manual refinement.

Discussion

The comparison demonstrates that AI-assisted programming can significantly improve software development efficiency by automating repetitive coding tasks and providing immediate implementation suggestions. The AI generated version required less effort to produce a functional application and included additional improvements such as enhanced documentation, descriptive console messages and input validation. These features contributed to improved readability and maintainability of the final program.

AI generated code was not adopted without modification. All generated classes were manually reviewed, tested and refined to ensure they satisfied the project requirements and produced the expected functionality. Human judgment remained essential for validating the correctness of the generated code, identifying logical issues and making design decisions. Consequently, the AI-assisted approach should be considered a collaborative development process rather than a complete replacement for traditional programming practices.

Task 3: Critical Analysis

3.1 How AI Changed the Coding Approach

The use of AI-assisted programming significantly changed the software development process compared to the traditional approach. Instead of manually writing every class and method from scratch, AI was used to generate an initial implementation based on natural language prompts. This allowed the development process to focus more on understanding the system requirements, validating the generated code and refining the implementation rather than writing repetitive code.

During the development of the Library Management System, Gemini assisted in generating the Book, Library, and Main classes, suggesting improvements to code structure and providing explanations for programming concepts when needed. The AI generated code served as a starting point rather than a final solution. Each generated component was manually reviewed, tested and modified to ensure that it met the project requirements and followed object oriented programming principles.

The development process therefore shifted from writing every line of code manually to supervising, evaluating, and improving AI generated solutions. This allowed more time to be spent on software design, debugging and testing while reducing the time required for repetitive implementation tasks.

3.2 Whether AI Improved or Reduced Understanding

The use of AI improved understanding in several aspects of software development. Gemini provided explanations of Java syntax, object oriented programming concepts and implementation techniques that helped clarify programming concepts encountered during development.

However, relying entirely on AI generated code without careful analysis could reduce a student's understanding of the underlying programming concepts. Simply copying generated code without reviewing its logic may prevent learners from developing essential problem solving and debugging skills. Therefore, AI should be considered a learning assistant rather than a replacement for independent programming practice.

3.3 Situations Where AI Should Not Be Used

Although AI-assisted programming provides many benefits, there are situations in which it should not be relied upon as the primary solution.

First, AI should not be used without verification in safety critical software systems such as medical devices, aviation software, financial transaction systems or autonomous vehicles. Errors in these applications may have serious consequences, making extensive human review and formal testing essential.

Second, AI should not replace the learning process in educational environments. Students who rely solely on AI generated code may complete programming tasks without developing the analytical thinking and problem solving skills expected from software developers.

AI should also be used cautiously when developing software involving confidential information or proprietary business logic. Entering sensitive source code or confidential project details into public AI systems may create privacy or security concerns depending on the organization's policies and the AI platform being used.

Finally, AI-generated code should not be accepted without testing or code review, as it may contain logical errors, inefficient algorithms, outdated programming practices or security vulnerabilities.

3.4 Ethical Considerations in AI-Generated Code

The increasing use of AI in software development raises several ethical considerations that developers must address responsibly.

One important consideration is transparency. Developers should clearly acknowledge when AI tools have been used during software development, particularly in academic assignments and professional projects. Proper disclosure promotes academic integrity and allows reviewers to understand the development process.

Another consideration is accountability. Although AI can generate working code, responsibility for the correctness, security and reliability of the final software remains with the developer. AI generated code should therefore be carefully reviewed, tested and validated before deployment.

Developers must also consider intellectual property and copyright issues. AI generated code may resemble existing programming patterns or publicly available examples, making it important to verify licensing requirements and avoid unintentional copyright infringement.

Privacy and data protection are additional ethical concerns. Developers should avoid submitting confidential source code, personal information, or sensitive organizational data to AI systems unless doing so complies with organizational policies and applicable data protection regulations.

Task 4: Conclusion

Artificial Intelligence is becoming an increasingly important tool in modern software development, offering significant benefits in both programming education and the software industry. This project demonstrated that AI-assisted programming can improve development efficiency by generating code, providing debugging assistance, suggesting refactoring opportunities and producing documentation. Compared with the traditional programming approach, the AI assisted implementation of the Library Management System required less effort for repetitive coding tasks while producing well structured and readable code.

From an educational perspective, AI can serve as an effective learning assistant by explaining programming concepts, demonstrating different implementation techniques and helping students understand compiler errors. However, we should use AI responsibly by reviewing, testing, and understanding the generated code rather than relying on it as a substitute for learning fundamental programming concepts. Developing analytical thinking, problem solving skills, and debugging abilities remains essential for becoming a competent software developer.

In the software industry, AI-assisted programming has the potential to increase developer productivity, improve code quality, and reduce development time. Nevertheless, AI generated code should always be subjected to thorough testing, code review and security validation before deployment. Human developers remain responsible for ensuring that software is accurate, reliable, secure and compliant with ethical and professional standards.

Overall, AI should be viewed as a powerful development assistant rather than a replacement for programmers. The most effective software development is achieved through collaboration between human expertise and AI capabilities, where AI automates routine tasks while developers provide critical thinking, creativity and professional judgment. This balanced approach enables organizations and learners to benefit from AI while maintaining high standards of software quality and ethical responsibility.

Project Resources

GitHub Repository :

<https://github.com/avishkaedi-arch/AI-Assisted-Library-Management-System>

Gemini Shared Conversation :

<https://share.gemini.google/l63wtac79ErW>